

Relatório - Atividade 08: Compilador EV

Equipe: Tiago Trindade de Oliveira; Ralf Dewrich Ferreira; Yan Fabber Lima de Albuquerque; Daniel Victor Carneiro Brandão da Costa.

1. O que foi feito

Foi implementado um compilador para a linguagem EV (Expressões com Variáveis). O programa recebe um arquivo .ev, reconhece declarações de variáveis e uma expressão final, valida o uso de identificadores declarados anteriormente e gera assembly x86-64 que calcula e imprime o resultado.

2. Como foi feito

A implementação foi organizada no arquivo compev.py, reaproveitando a estrutura do compilador EC2 da Atividade 07. O analisador léxico reconhece números, identificadores ASCII, operadores, parênteses, sinal de igual e ponto-e-vírgula. O analisador sintático descendente recursivo produz uma AST com o nó raiz Programa, uma lista de declarações e a expressão de resultado.

Depois da análise sintática, a etapa semântica percorre as declarações em ordem com uma tabela de símbolos. Cada expressão de declaração só pode usar variáveis já declaradas; ao fim da verificação, o nome declarado entra na tabela. A expressão final também é verificada contra essa tabela.

Na geração de código, cada variável é reservada na seção .bss com uma diretiva .lcomm. O valor de cada declaração é calculado em %rax e armazenado no símbolo da variável. Referências a variáveis carregam esse valor de volta para %rax. O código gerado chama imprime_num e sair, definidos em runtime.s.

```
x = (7 + 4) * 12;  
y = x * 3 + 11;  
= (x * y) + (x * 11) + (y * 13)
```

3. Arquivos entregues

Arquivo	Descrição
compev.py	Código-fonte do compilador EV.
runtime.s	Rotinas de apoio para imprimir o resultado e encerrar o programa.
Dockerfile	Ambiente com Python e binutils para executar a suíte completa.
README.md	Documentação de uso, gramática, arquitetura e testes.
plano_relatorio_atv8.txt	Plano e relatório sucintos da sessão de implementação.
relatorio_compilador_ev.pdf	Este relatório em PDF.
test.sh	Script de testes com oráculo e verificação cruzada.
testes/*.ev	Entradas válidas e inválidas da linguagem EV.
testes/esperado/*.out	Saídas esperadas dos casos válidos.

4. Como executar

Compilar um programa EV para assembly:

```
python3 compev.py testes/v4_dependencias.ev -o prog.s
```

Montar, linkar e executar o assembly gerado em Linux x86-64:

```
as --64 -o prog.o prog.s  
ld -o prog prog.o  
./prog
```

Executar a suíte de testes localmente:

```
bash test.sh
```

Executar a suíte completa em Docker:

```
docker build -t compev .  
docker run --rm -v "$PWD":/app compev
```

5. Testes realizados

Foram criados 21 testes automatizados: 10 válidos e 11 inválidos. Para cada caso válido, o script gera o assembly, compara o resultado do interpretador (--avaliar) com o arquivo esperado e, quando há toolchain x86-64 funcional, monta, linka e executa o binário para comparar a saída real.

Grupo	Cobertura	Resultado esperado
Válidos	Resultado sem declarações, variáveis simples, dependências entre declarações, precedência, associatividade, divisão truncada e identificadores com letras/dígitos.	Compilação aceita; interpretador, golden output e binário concordam.
Inválidos	Variável não declarada, identificador começando por dígito, falta de ponto-e-vírgula, ausência de resultado, operador unário, divisão por zero, token extra, caractere inválido, expressão incompleta e dígito Unicode.	Compilador rejeita a entrada com código de saída diferente de zero.

6. Conclusão

O projeto atende aos requisitos da Atividade 08: implementa a linguagem EV, inclui análise semântica para variáveis não declaradas, gera assembly com variáveis na seção .bss, fornece documentação objetiva e possui testes válidos e inválidos. A validação foi executada localmente e também em Docker, com montagem, linkedição e execução dos binários gerados.